

3.2.6 Cross Site Request Forgery (CSRF)

“A CSRF attack forces a logged-on victim’s browser to send a forged HTTP request, including the victim’s session cookie and any other automatically included authentication information, to a vulnerable web application. This allows the attacker to force the victim’s browser to generate requests the vulnerable application thinks are legitimate requests from the victim.”

For most web attacks authentication is important. The attacker must disguise the attack as if it is an authenticated / authorized request. The genuineness of the attack makes it difficult to detect and block.

What if then, you could have the end-user himself do your dirty deed for you. Instead of trying to impersonate a user, or bypass the authentication system, how about using the user to conduct the attack on himself / herself?

In simple words, why bother to forge a signature when you can forge the amount with greater ease, when you have a blank check ready-signed.

A Cross-site request forgery does exactly that. Instead of forging the user credentials, you forge the request that will get you what you need, and somehow have the user perform the request. What CSRF relies on is that any request sent by a logged in user will be considered legitimate, and the web site will have no way of blocking it.

For a transaction conducted on a money transfer website such as PayPal, how is the website to tell if the user really wanted to transfer funds to the hacker, or whether it was an attack if it is coming from a legitimate authorized user. After all, transferring funds is a feature of the website, and they expect such usage from their users.

You might be wondering how exactly such an attack would be constructed. What we have explained till now doesn’t really cover it. Despite the seeming ambiguity of such attacks, they are actually quite simple. Don’t worry; the following example will make it very clear:

Let us take an example of an online multi-player strategy game. The game – as usual for its genre – involves gathering resources such as wood and gold. Now suppose that an attacker discovers that when he transfers gold to another player in the game, the URL request looks like the following:

`http://awesome.strategygame.com/transfer.php?item=gold&user=sumguy&amount=10000`

Now he can’t change the sender of the money, only the person who will receive the money, and the amount. As such changing the URL will not result in the user being able to do anything they cannot already do with the interface provided. What they need to do is to somehow get other users to visit this URL while logged in, and have it transfer money to the hackers account.

What the hacker now does is posts some specially constructed HTML code in his profile page, or in the forum section for the game, or anywhere